Computer Graphics Notes

For BCA 4 Year Program

Prepared by:- Damanjeet Kaur

Unit 1

Overview of computer graphics and its applications, Historical development, Application areas e.g. entertainment, scientific visualization, user interfaces, Basics of raster and vector graphics, differences between raster and vector graphics, Representation and storage formats, Graphics pipeline and rendering process, stages of the graphics pipeline (modeling, transformation, rendering), concepts of rendering primitives, vertices, and fragments.

Computer graphics (CG) is the field of study that focuses on the creation, manipulation, and storage of geometric objects and images using computers

. It serves as a vital communication bridge, allowing data to be represented visually for easier human interpretation.

Historical Development

The evolution of CG began in the **1950s** with military projects like **Whirlwind** and **SAGE**, which used Cathode Ray Tubes (CRTs) and light pens for interaction.

- **1960s**: Ivan Sutherland's **Sketchpad (1963)** revolutionized interactive graphics. The term "computer graphics" was coined at Boeing in 1960.
- **1970s**: Foundational algorithms for **shading** (Phong, Gouraud) and the first **framebuffers** for raster graphics emerged.
- **1980s**: Personal computers like the **Macintosh** brought GUIs to consumers, and the first **GPUs** (like NEC's) were developed.
- **1990s-Present**: 3D graphics went mainstream with **Toy Story (1995)**, standards like **OpenGL/DirectX**, and the rise of real-time ray tracing in the late 2010s.

Application Areas
Computer graphics is integral across various sectors:

- **Entertainment**: Used for CGI in movies (special effects, animation) and interactive real-time graphics in video games.
- **Scientific Visualization**: Helps researchers visualize complex datasets, such as molecular structures, weather patterns, and medical MRI/CT scans.
- **User Interfaces (UI)**: Provides the graphical elements (icons, windows, menus) that make software intuitive and accessible.

- **Computer-Aided Design (CAD)**: Enables engineers and architects to create precise 3D models of buildings and machinery.

Raster vs. Vector Graphics
The two primary ways of representing digital images differ fundamentally in their composition and scalability.

Feature	Raster Graphics (Bitmap)	Vector Graphics
Composition	Made of a grid of tiny colored dots called **pixels**.	Defined by **mathematical formulas**, lines, and curves.
Scalability	Becomes pixelated or "jagged" when enlarged (aliasing).	Infinitely scalable without any loss of quality or clarity.
Detail	Ideal for high-detail photos and subtle color gradations.	Best for logos, icons, and geometric illustrations.
File Size	Generally larger; size grows with resolution/pixel count.	Much smaller; only stores mathematical data for paths.
Formats	**JPEG, PNG, GIF, BMP, TIFF**.	**SVG, PDF, AI, EPS**.

The Graphics Pipeline & Rendering Process

The graphics pipeline is the sequence of steps a GPU follows to transform 3D data into a 2D image on a screen.

1. **Modeling (Application Stage)**: The CPU defines the scene, including the geometry of objects, light sources, and camera position.
2. **Transformation (Geometry Stage)**:

1. **Vertices**: The basic points in 3D space.
2. **Vertex Processing**: Vertices are transformed from their local "object space" to "world space" and finally "screen space" using mathematical matrices.
3. **Rendering (Rasterization & Shading)**:

0. **Primitives**: Basic shapes (usually triangles) formed by connecting vertices.

1. **Rasterization**: The GPU determines which screen pixels are covered by each primitive.

2. **Fragments**: These are "potential pixels" generated during rasterization, containing data like color, depth, and texture.

3. **Fragment/Pixel Shading**: The final color of each fragment is calculated, often using textures and lighting models, before being written to the framebuffer for display.

Primitives

In computer graphics, **graphics primitives** are the fundamental geometric building blocks used to construct complex scenes and images.

1. Types of Primitives

Primitives are low-level objects from which higher-level graphics are built. The most common types include:

- **Lines**: Straight paths connecting two points (vertices). They are used for wireframe models, charts, and outlines.
- **Polygons**: Closed figures made of three or more line segments (edges) joined at vertices. **Triangles** are the most common primitive used in modern GPUs because they are always planar and easy to rasterize.
- **Points**: Represent a single position in space defined by coordinates (e.g. X,Y in 2D). On a screen, a point is often represented by a single pixel.

2. Coordinate Systems

Coordinate systems provide a mathematical way to assign numbers to points in space.

Cartesian Coordinate System

This is the standard (3D) system based on perpendicular axes. Points are defined by their signed distances from these axes.

- **Pros**: Highly intuitive for measuring physical distances.
- **Cons**: Certain transformations, like translation (moving an object), cannot be represented as simple matrix multiplications; they require vector addition

Homogeneous Coordinate System

This system adds an extra dimension (typically called X,Y) to represent points. A 2D point becomes (X,Y) , and a 3D point becomes (X,Y,Z)

- **Unified Transformations**: Allows all transformations (translation, rotation, scaling) to be performed using only matrix multiplication
- **Points at Infinity**: Can represent infinitely distant points, which is crucial for perspective projections.

- **Matrix Concatenation**: Multiple transformations can be "baked" into a single matrix by multiplying them together, significantly reducing computational overhead during rendering.

3. Transformation Matrices & Applications

Transformation matrices are

Transformation	Purpose	Mathematical Application
Translation	Shifts an object to a new location.	Moves a model to its correct world position.
Scaling	Changes the size of an object.	Uniform or non-uniform resizing of models.
Rotation	Pivots an object around an axis.	Orienting a character to face a specific direction.
Shearing	Slants an object by shifting its vertices proportionally.	Creating "italic" text or simulated 2D depth.
Projection	Maps 3D coordinates onto a 2D screen.	Simulates perspective (objects getting smaller with distance).

Unit 2

In 2D graphics programming, drawing algorithms and pixel operations bridge the gap between abstract geometry (like a line equation) and the physical grid of pixels on a screen.

1. Color Models

Color models provide a mathematical way to represent colors as tuples of numbers.

- **RGB (Red, Green, Blue)**: An **additive** model used for digital displays. Combining all three at max intensity creates white.
- **CMYK (Cyan, Magenta, Yellow, Black)**: A **subtractive** model used for color printing. Combining the first three creates a dark muddy brown, so "Key" (black) is added for true depth.
- **HSL (Hue, Saturation, Lightness)**: A **perceptual** model that is more intuitive for humans. It separates the "color" (Hue) from its "vibrancy" (Saturation) and "brightness" (Lightness).

2. Pixel Operations

Once a pixel's base color is determined, it can be modified:

- **Blending (Alpha Blending)**: Combines a new "source" color with an existing "destination" color using an **alpha** (transparency) value.
- **Interpolation**: Estimating color values between known points. **Linear interpolation (Lerp)** is used to create smooth gradients across a line or surface.

---

3. Line Drawing Algorithms

Since screens are discrete grids, drawing a "straight" line requires choosing which pixels best approximate the path.

DDA (Digital Differential Analyzer)

A floating-point based approach. It calculates the unit change in and (the slope) and increments the coordinates step-by-step.

- **Pros**: Very simple to understand and implement.
- **Cons**: Requires **floating-point arithmetic** and rounding at every step, which is computationally expensive for older or low-power hardware.

Bresenham's Line Algorithm

An optimized algorithm that uses **integer-only arithmetic**. It uses a "decision variable" to determine if the next pixel should be horizontal or diagonal.

- **Pros**: Extremely fast; no divisions or floats.
- **Cons**: Slightly more complex logic than DDA.

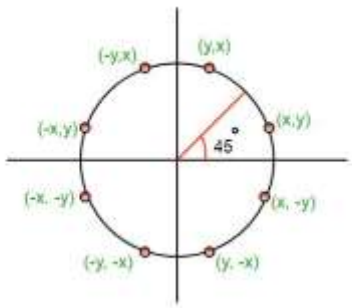
---

4. Circle Drawing Algorithms

Drawing circles is harder because the "slope" changes constantly.

Midpoint Circle Algorithm

Similar to Bresenham's, it uses a decision parameter to see if a pixel is inside or outside the theoretical circle boundary. It calculates coordinates for only **one-eighth (an octant)** of the circle and uses 8-way symmetry to mirror the points to the rest of the circle.



$P_k = (X_k — 0.5)^2 + (y_k + 1)^2 - r^2$

Now,

$x_{k+1} = x_k$ or x_{k-1} , $y_{k+1} = y_k + 1$

$P_{k+1} = (x_{k+1} – 0.5)^2 + (y_{k+1} + 1)^2 - r^2$

$= (x_{k+1} – 0.5)^2 + [(y_k + 1) + 1]^2 - r^2$

$= (x_{k+1} – 0.5)^2 + (y_k + 1)^2 + 2(y_k + 1) + 1 – r^2$

$= (x_{k+1} – 0.5)^2 + [- (x_k - 0.5)^2 + (x_k - 0.5)^2] + (y_k + 1)^2 – r^2 + 2(y_k + 1) + 1$

$= P_k + (x_{k+1} - 0.5)^2 - (x_k - 0.5)^2 + 2(y_k + 1) + 1$

$= P_k + (x^2_{k+1} – x^2_k) - (x_{k+1} - x_k) + 2(y_k + 1) + 1$

$= P_k + 2(y_k + 1) + 1$, when $P_k <= 0$ i.e the midpoint is inside the circle

$(x_{k+1} = x_k)$

$P_k + 2(y_k + 1) – 2(x_k - 1) + 1$, when $P_k > 0$ I.e the mid point is outside the circle$(x_{k+1} = x_k - 1)$

Bresenham's Circle Algorithm

Nearly identical to the Midpoint method in practice, it focuses on minimizing the distance between the pixel and the true circular arc using integer addition and subtraction.

Algorithm Steps (for a circle centered at the origin)

The algorithm generates points from the initial position (0, r) in the first quadrant until the x-coordinate is greater than or equal to the y-coordinate ($x \geq y$).

1. **Input** the radius r and the circle's center coordinates (xc, yc).

2. **Initialize** the starting point as $x = 0$, $y = r$.

3. **Calculate** the initial value of the decision parameter d (or p) as $d = 3 - 2r$.

4. **Plot** the initial point and its seven symmetric points using the eight-way symmetry property:

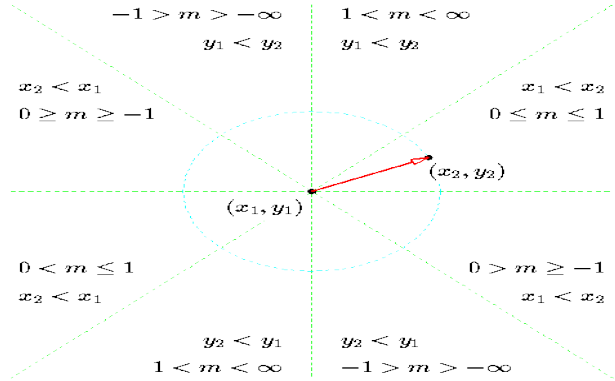
- $(xc + x, yc + y)$
- $(xc - x, yc + y)$
- $(xc + x, yc - y)$
- $(xc - x, yc - y)$
- $(xc + y, yc + x)$
- $(xc - y, yc + x)$
- $(xc + y, yc - x)$
- $(xc - y, yc - x)$

5. **Iterate** the following steps in a loop while $x < y$:

- Increment x by 1 ($x = x + 1$).
- **Check** the value of the decision parameter d:
  - If $d < 0$, the next pixel is closer to the East, so update d as $d = d + (4 * x) + 6$.
  - If $d \geq 0$, the next pixel is closer to the South-East, so update d as $d = d + 4 * (x - y) + 10$ and decrement y by 1 ($y = y - 1$).
- **Plot** the current (x, y) point and its seven symmetric points (as described in step 4), adjusting for the center (xc, yc).

Advantages

- **Efficiency**: It uses only integer addition, subtraction, and bit shifting, making it very fast compared to algorithms using floating-point math, square roots, or trigonometry.
- **Simplicity**: The algorithm is relatively simple to implement.
- **Accuracy**: It generally produces circles that closely approximate true circles using discrete pixels.



Basic 2D Transformations

2D Transformations in computer graphics involve manipulating points, lines, or polygons in a two-dimensional plane. The basic transformations include **Translation** (moving), **Rotation** (rotating), and **Scaling** (resizing), while **Mirror Reflection** creates a reversed copy across an axis. To efficiently handle complex manipulations, these operations are combined into a single matrix using **homogeneous coordinates** (3x3 matrices).

These are the fundamental operations, usually represented by 2x2 or 3x3 matrices.

- Translation
- Scaling
- Shearing
- Rotation
- Reflection

- **Homogeneous Coordinates:** Used to multiply translation with rotation/scaling matrices, usually by adding a dummy coordinate '1' (3x3 matrix).

<u>Explain Viewing transformation pipeline</u>

- Converting world co-ordinates to viewing co-ordinates.
- Normalizing viewing co-ordinates.
- Converting normalized viewing co-ordinates to device co-ordinates.

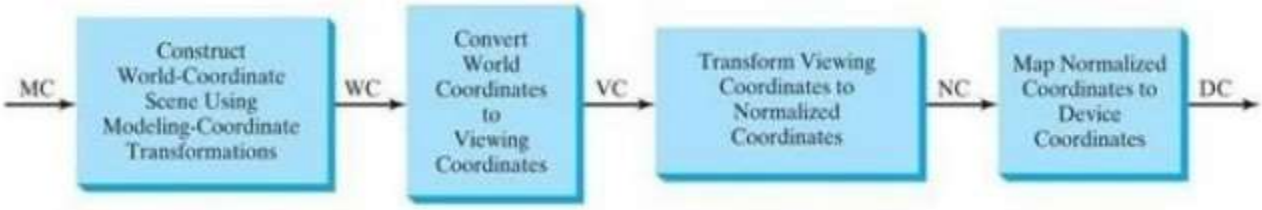


Fig. (c) Two-dimensional viewing transformation pipeline

In computer graphics, windowing techniques determine which part of a vast virtual world is selected for display and how it appears on your physical screen.

Core Terminologies

- **Window (World-Coordinate Window)**: A rectangular area in the **world coordinate system** that selects a specific portion of a scene for viewing. It essentially defines **what** you want to see.
- **Viewport**: A rectangular area on a **display device** (like your monitor) where the contents of the window are mapped. It defines **where** on the screen the image will appear.
- **Clipping Window**: The actual boundary against which objects are checked for visibility. Any part of an object (lines, polygons, etc.) that lies outside this rectangle is "clipped" or discarded so it isn't rendered.

Unit 3

Core Terminologies

- **Window (World-Coordinate Window)**: A rectangular area in the **world coordinate system** that selects a specific portion of a scene for viewing. It essentially defines **what** you want to see.
- **Viewport**: A rectangular area on a **display device** (like your monitor) where the contents of the window are mapped. It defines **where** on the screen the image will appear.
- **Clipping Window**: The actual boundary against which objects are checked for visibility. Any part of an object (lines, polygons, etc.) that lies outside this rectangle is "clipped" or discarded so it isn't rendered.

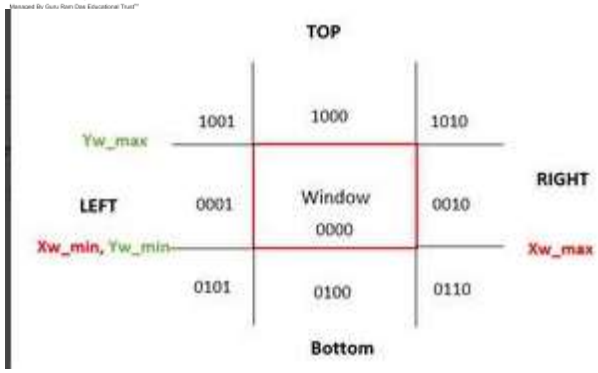
Region Codes (Cohen-Sutherland Algorithm)

To efficiently determine if a line needs to be clipped, the 2D space around the clipping window is divided into **8 regions**. Each region is assigned a unique **4-bit binary code** (often called an outcode), typically following the **TBRL** (Top, Bottom, Right, Left) format.

Bit	Position	Condition
Bit 1 (T)	Top	Set to 1 if the point is above the window
Bit 2 (B)	Bottom	Set to 1 if the point is below the window
Bit 3 (R)	Right	Set to 1 if the point is to the right of the window
Bit 4 (L)	Left	Set to 1 if the point is to the left of the window

Visibility Tests using Region Codes:

- **Trivial Acceptance**: If both endpoints of a line have the code `0000`, the line is entirely inside the window and is drawn.
- **Trivial Rejection**: If the bitwise **AND** of the two endpoint codes is not `0000`, both points are outside on the same side (e.g., both are above the window), and the line is discarded.
- **Clipping Required**: If neither of the above is true, the line is partially visible or potentially visible, and the algorithm must calculate intersections with the window boundaries.



Clipping Algorithm

Steps

1) Assign the region codes to both endpoints.

2) Perform OR operation on both of these endpoints.

3) if OR = 0000,

then it is completely visible (inside the window).

else

Perform AND operation on both these endpoints.

i) if AND ? 0000,

then the line is invisible and not inside the window. Also, it **can't be** considered for clipping.

ii) else

AND = 0000, the line is partially inside the window and **considered for clipping.**

4) After confirming that the **line is partially inside** the window, then we find the intersection with the boundary of the window. By using the following formula:-

Slope:- m= (y2-y1)/(x2-x1)

a) If the line passes through **top** or the line intersects with the top boundary of the window.

x = x + (y_wmax - y)/m

y = y_wmax

b) If the line passes through the **bottom** or the line intersects with the bottom boundary of the window.

x = x + (y_wmin - y)/m

y = y_wmin

c) If the line passes through the **left** region or the line intersects with the left boundary of the window.

y = y+ (x_wmin - x)*m

x = x_wmin

d) If the line passes through the **right** region or the line intersects with the right boundary of the window.

y = y + (x_wmax -x)*m
x = x_wmax

5) Now, overwrite the endpoints with a new one and update it.

6) Repeat the 4th step till your line doesn't get completely clipped

3D Graphics Programming

These are the fundamental operations, usually represented by 3x3 matrices.

- Translation
- Scaling
- Shearing
- Rotation
- Reflection

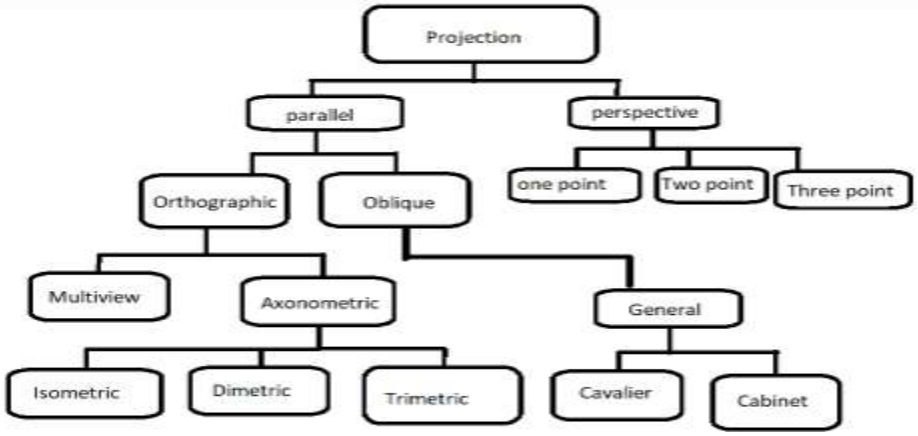
$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix}$	$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$Sh = \begin{bmatrix} 1 & sh_x^y & sh_x^z & 0 \\ sh_y^x & 1 & sh_y^z & 0 \\ sh_z^x & sh_z^y & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Translation Matrix	**Scaling Matrix**	**Shear Matrix**
$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos\theta & -sin\theta & 0 \\ 0 & sin\theta & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$R_y(\theta) = \begin{bmatrix} cos\theta & 0 & sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -sin\theta & 0 & cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$R_z(\theta) = \begin{bmatrix} cos\theta & -sin\theta & 0 & 0 \\ sin\theta & cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Rotation Matrix		

Why use 4x4 Matrices in 3D?

1. **Unified Pipeline:** All operations (moving, resizing, spinning) use the same mathematical function.
2. **Efficiency:** Hardware (GPUs) is specifically designed to perform 4x4 matrix multiplications at incredible speeds.

3. **Perspective:** The coordinate is essential for **Perspective Projection**, making distant objects appear smaller.

Projection



Orthographic and perspective projections are two fundamental methods for representing 3D objects on a 2D plane. Each serves different purposes in engineering drawing; below are the key differences, strengths, and typical uses.

Core definitions

- Orthographic projection: Parallel projection where projection rays are parallel to each other and perpendicular (or at known angles) to the projection plane. Common forms: orthogonal front/plan/side views and auxiliary views.
- Perspective projection: Converging projection where projection rays meet at a single point (the station or viewpoint), producing images that mimic human visual perception with foreshortening and vanishing points.

Geometric differences

- Rays: Orthographic — parallel; Perspective — converge to a viewpoint.
- Scale: Orthographic — true scale (along directions parallel to projection) for each view; Perspective — scale varies with depth (objects farther from viewpoint appear smaller).
- Vanishing points: Orthographic — none (parallel edges remain parallel); Perspective — one or more vanishing points where parallel lines converge.

- Foreshortening: Orthographic — controlled and predictable per view; Perspective — continuous depth foreshortening across the image.

Accuracy and measurement

- Measurements: Orthographic views allow direct measurement and dimensioning (lengths, angles) in the plane without correction. Perspective images do not permit direct accurate measurement of true dimensions.
- Representing true shapes: Orthographic projections can show true shape and size of faces when the viewing direction is normal to the face. Perspective cannot show true-size faces unless corrected.

Typical uses in engineering

- Orthographic:
  - Technical documentation, manufacturing drawings, inspection, CAD-produced working drawings.
  - Communicating exact dimensions, tolerances, and geometric relationships.
- Perspective:
  - Visualisation, concept presentations, marketing renderings, architectural massing studies.
  - Giving a realistic sense of depth and spatial appearance for non-manufacturing purposes.

Variants and standards

- Orthographic variants: First-angle and third-angle projection standards dictate placement of views (regional standardization: Europe largely first-angle, North America third-angle).
- Perspective variants: One-point, two-point, and three-point perspective depending on orientation of edges relative to viewer.

Practical implications for engineers and drafters

- Use orthographic projections for fabrication, inspection, and precise communication of geometry and tolerances.
- Use perspective for client presentations, human-centric visualization, and when conveying aesthetic or spatial context.
- Many CAD systems produce both: generate orthographic views for drawings and perspective views for renderings.

Summary table (concise)

- Projection rays: orthographic = parallel; perspective = convergent.
- Scale: orthographic = uniform/true on each view; perspective = non-uniform with depth.

- Measurement: orthographic = directly measurable; perspective = not directly measurable.
- Purpose: orthographic = technical accuracy; perspective = realistic visualization.

Note on practice: Engineering deliverables typically combine both—orthographic drawings for manufacturing and legal documentation, perspective renderings for stakeholder communication.

Unit 4

Lighting and shading models

Illumination model, also known as Shading model or Lighting model, is used to calculate the intensity of light that is reflected at a given point on surface. There are three factors on which lighting effect depends on:

1. **Light Source :** Light source is the light emitting source. There are three types of light sources: Their position, electromagnetic spectrum and shape determine the lighting effect.
    1. **Point Sources -** The source that emit rays in all directions (A bulb in a room).
    2. **Parallel Sources -** Can be considered as a point source which is far from the surface (The sun).
    3. **Distributed Sources -** Rays originate from a finite area (A tubelight).
2. **Surface :** When light falls on a surface part of it is reflected and part of it is absorbed. Now the surface structure decides the amount of reflection and absorption of light. The position of the surface and positions of all the nearby surfaces also determine the lighting effect.
3. **Observer :** The observer's position and sensor spectrum sensitivities also affect the lighting effect

1. Ambient Illumination

Assume you are standing on a road, facing a building with glass exterior and sun rays are falling on that building reflecting back from it and the falling on the object under observation. This would be **Ambient Illumination,**In simple words, Ambient Illumination is the one where source of light is indirect. The reflected intensity I_{amb} of any point on the surface is:

$$I_{amb} = K_a I_a$$

Where, I_a : ambient light intensity

K_a : surface ambient reflectivity, value of K_a varies from 0 to 1

2. Diffuse Reflection

Diffuse reflection occurs on the surfaces which are rough or grainy. In this reflection the brightness of a point depends upon the angle made by the light source and the surface. The reflected intensity I_{diff} of a point on the surface is:

$$I_{diff} = K_d I_p \cos(\theta) = K_d I_p (N \cdot L)$$

Where, I_p : the point light intensity
K_d : the surface diffuse reflectivity, value of K_d varies from 0 to 1
N : the surface normal
L : the light direction

3. Specular Reflection

When light falls on any shiny or glossy surface most of it is reflected back, such reflection is known as Specular Reflection. **Phong Model** is an empirical model for Specular Reflection which provides us with the formula for calculation the reflected intensity $I_{spec.}$

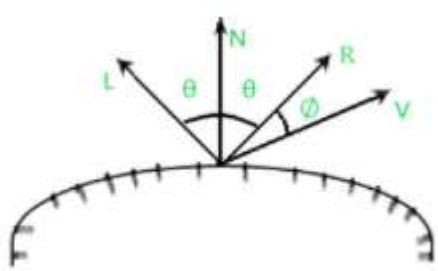
$$I_{spec} = W(\theta)\, I_l \cos^n(\Phi)$$

where, $W(\theta)$: K_s
L : direction of light source
N : normal to the surface
R : direction of reflected ray
V : direction of observer
Θ : Angle between L and R
Φ : angle between R and V

Phong model of reflection :

When we look at illuminated shiny surfaces, such as glittering surfaces, polished metal sheets, apple etc, we found a kind of bright spot at certain viewing point locations. This phenomenon is called specular reflection.

Look at the following figure :



N = Normal vector

L = Point light source

V = Viewing direction

R = is representing the unit vector directed towards the ideal specular reflection

Ø = Viewing angle relative to the specular reflection direction R.

θ = Angle made by L & R with N.

For ideal reflector surfaces(perfect mirror), incident light is reflected only in the specular-reflection direction. So, in this case, we could be able to see reflected light when **vectors V & R coincides(viewing angle(Ø=0))**.

Polygon Rendering Methods

Below are the main techniques used to render polygon surfaces:

1. Constant Intensity (Flat) Shading

Description:

- Each polygon surface is rendered with a **single color** (constant intensity).
- Lighting is calculated **once per polygon**, using the surface normal.

Features:

- Fast and simple.
- Suitable for flat surfaces.

Limitations:

- Causes **Mach banding** (visible boundaries between polygons).
- Unrealistic for curved or smooth objects.

Use case: Low-end graphics, CAD wireframes, or real-time previews.

2. Gouraud Shading (Intensity Interpolation)

Description:

- Calculates lighting at **each vertex**.
- The vertex intensities are **interpolated** across the surface of the polygon.

Steps:

1. Compute illumination at vertices.
2. Linearly interpolate intensity across edges.

3. Interpolate across scanlines (inside the polygon).

Advantages:

- Smooth shading across surfaces.
- Removes sharp intensity edges.

Limitations:

- May miss specular highlights (shiny spots) between vertices.

Use case: Games, 3D animation, low-cost rendering.

3. Phong Shading (Normal Vector Interpolation)

Description:

- Interpolates **surface normals** instead of intensities.
- Computes lighting **per pixel**, giving more accurate highlights.

Steps:

1. Calculate normals at each vertex.
2. Interpolate normals across polygon.
3. Compute lighting for each pixel.

Advantages:

- Very smooth, realistic shading.
- Captures specular reflections accurately.

Limitations:

- More computationally expensive than Gouraud.

Use case: High-quality rendering, animation, ray tracing.

Hidden surface removal algorithms:

Z-buffer algorithm

Z-buffer, which is also known as the Depth-buffer method is one of the commonly used method for hidden surface detection. It is an Image space method. Image space methods are based on the pixel to be drawn on 2D. For these methods, the running time complexity is the number of pixels times number of objects.

Z values are stored in 2 buffers

1. Z buffer
2. Refresh buffer

Image space method that remove hidden surface , it compares surface depth at each pixel.

Z-Buffer (Depth Buffer) Algorithm

The Z-buffer algorithm is the most widely used technique, largely due to its simple implementation in hardware.

- **Approach:** It is an image-space method, determining visibility at each pixel location.
- **Data Structures:** It uses two buffers:
  - **Frame buffer:** Stores the final color value for each pixel (x, y).
  - **Depth buffer (Z-buffer):** Stores the depth (z-value) of the surface closest to the viewer found so far at each pixel (x, y). Initial values are typically set to maximum depth or infinity.
- **Process:**

1. Initialize the depth buffer with a large value and the frame buffer with the background color.
2. For each polygon in the scene, the algorithm iterates through all pixels it covers.
3. At each pixel, the current depth of the polygon is calculated and compared with the depth value already stored in the Z-buffer.

4. If the new depth value is smaller (closer to the viewer), the frame buffer is updated with the polygon's color, and the depth buffer value is replaced with the new, smaller depth.

- **Advantages:** Simple to implement, handles complex scenes with overlapping polygons effectively, and polygons can be processed in any order.
- **Disadvantages:** Requires significant extra memory for the depth buffer and can suffer from precision errors (z-fighting).

```
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.Arrays;

public class ZBufferRenderer {
    private int width, height;
    private float[] zBuffer;
    private BufferedImage frameBuffer;

    public ZBufferRenderer(int width, int height) {
        this.width = width;
        this.height = height;
        this.frameBuffer = new BufferedImage(width, height,
BufferedImage.TYPE_INT_RGB);
        this.zBuffer = new float[width * height];
        clearBuffers();
    }

    public void clearBuffers() {
        // Initialize Z-buffer with maximum depth (infinity)
        Arrays.fill(zBuffer, Float.POSITIVE_INFINITY);

        // Clear frame buffer to black
        Graphics2D g = frameBuffer.createGraphics();
        g.setColor(Color.BLACK);
        g.fillRect(0, 0, width, height);
        g.dispose();
    }

    public void drawTriangle(int x1, int y1, float z1, int x2, int y2, float z2, int x3, int y3, float
z3, int color) {
        // Simple bounding box for the triangle
```

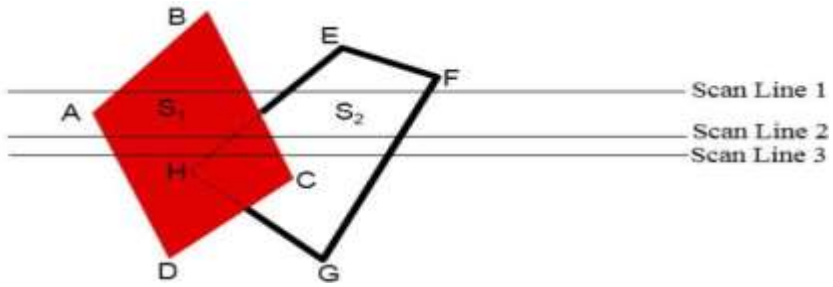
```
        int minX = Math.max(0, Math.min(x1, Math.min(x2, x3)));
        int maxX = Math.min(width - 1, Math.max(x1, Math.max(x2, x3)));
        int minY = Math.max(0, Math.min(y1, Math.min(y2, y3)));
        int maxY = Math.min(height - 1, Math.max(y1, Math.max(y2, y3)));

        for (int y = minY; y <= maxY; y++) {
            for (int x = minX; x <= maxX; x++) {
                if (isInsideTriangle(x, y, x1, y1, x2, y2, x3, y3)) {
                    // Interpolate Z-depth at (x, y) using barycentric coordinates
                    float currentZ = interpolateZ(x, y, x1, y1, z1, x2, y2, z2, x3, y3, z3);

                    int index = y * width + x;
                    // Depth Test
                    if (currentZ < zBuffer[index]) {
                        zBuffer[index] = currentZ;
                        frameBuffer.setRGB(x, y, color);
                    }
                }
            }
        }
    }

    // Standard point-in-triangle test (cross-product method)
    private boolean isInsideTriangle(int px, int py, int x1, int y1, int x2, int y2, int x3, int y3) {
        // Implementation of barycentric or edge-function test
        return true; // Simplified placeholder
    }

    private float interpolateZ(int px, int py, int x1, int y1, float z1, int x2, int y2, float z2, int
x3, int y3, float z3) {
        // Linear interpolation logic across the triangle face
        return (z1 + z2 + z3) / 3.0f; // Simplified placeholder
    }
}
```

Scan-Line Method

It is an image-space method to identify visible surface. This method has a depth information for only single scan-line. In order to require one scan-line of depth values, we must group and process all polygons intersecting a given scan-line at the same time before processing the next scan-line. Two important tables, **edge table** and **polygon table,** are maintained for this.

The Edge Table − It contains coordinate endpoints of each line in the scene, the inverse slope of each line, and pointers into the polygon table to connect edges to surfaces.

The Polygon Table − It contains the plane coefficients, surface material properties, other surface data, and may be pointers to the edge table.



To facilitate the search for surfaces crossing a given scan-line, an active list of edges is formed. The active list stores only those edges that cross the scan-line in order of increasing x. Also a flag is set for each surface to indicate whether a position along a scan-line is either inside or outside the surface.

Pixel positions across each scan-line are processed from left to right. At the left intersection with a surface, the surface flag is turned on and at the right, the flag is turned off. You only need to perform depth calculations when multiple surfaces have their flags turned on at a certain scan-line position.

Scan Line Algorithm for Hidden Surface Removal

The Scan line algorithm is an extension of the basic polygon filling algorithm and also operates in image space, but it uses coherence to optimize processing.

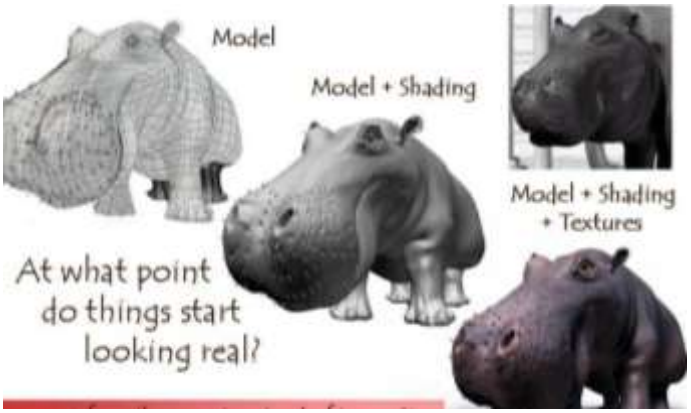
- **Approach:** It processes the scene one horizontal scan line at a time, considering all polygons that intersect the current line.
- **Data Structures:** It maintains an Edge Table, an Active Edge List, and a Polygon Table to manage edge information, active edges for the current scan line, and polygon properties (like the plane equation and color).
- **Process:**

1. For each scan line, it finds all intersections with polygon edges and sorts them by x-coordinate.

2. It then scans horizontally between these intersections (spans).
3. For overlapping polygons within a span, it calculates and compares their z-values (depths) at each pixel to determine the closest (visible) one.
4. The frame buffer is updated with the color of the visible polygon.

- **Advantages:** More memory efficient than the Z-buffer because depth information is only needed for a single scan line at a time. It effectively uses coherence properties to speed up calculations.
- **Disadvantages:** More complex to implement than the Z-buffer algorithm and requires more complex data structures and bookkeeping to handle all polygons intersecting a scan line simultaneously. It struggles with non-polygonal or non-planar surfaces.

```
// Logic for a single scan line iteration
public void processScanLine(int y) {
    updateActiveEdgeTable(y);
    Collections.sort(activeEdgeTable); // Sort by current x-intercept

    for (int i = 0; i < activeEdgeTable.size() - 1; i++) {
        int xStart = activeEdgeTable.get(i).xIntercept;
        int xEnd = activeEdgeTable.get(i+1).xIntercept;

        // Update Polygon Table flags based on edges encountered
        togglePolygonFlag(activeEdgeTable.get(i).polygonID);

        for (int x = xStart; x < xEnd; x++) {
            List<Polygon> activePolys = getPolygonsWithFlagOn();
            if (activePolys.isEmpty()) {
                frameBuffer[x][y] = backgroundColor;
            } else {
                Polygon closest = findClosestPolygon(activePolys, x, y);
                frameBuffer[x][y] = closest.color;
            }
        }
    }
}
```



Texture Coordinates and Mapping Techniques

Texture mapping is the process of "pasting" a 2D image (the texture) onto a 3D object's surface.

- **Texture Coordinates (u,v):** These are 2D coordinates used to map a specific point on a texture image to a vertex on a 3D model.
  - The origin(0,0) is usually the lower-left corner(1,1), and is the upper-right.
  - During rendering, these coordinates are interpolated across the surface of a polygon to determine which **texel** (texture pixel) corresponds to each screen pixel.
- **Mapping Techniques:**
  - **Planar Mapping:** Projects the texture onto a flat surface from one direction.
  - **Cylindrical/Spherical Mapping:** Wraps the texture around the object like a label on a bottle or a skin on a ball.
  - **Wrap Modes:** Defines what happens when coordinates fall outside the [0,1] range, such as **Clamping** (stretching the edge pixel) or **Wrapping** (repeating the texture).

2. Texture Filtering

Texture filtering determines how a color is calculated when a screen pixel doesn't perfectly align with a single texel—common when zooming in (magnification) or moving away (minification).

- **Nearest-Neighbour Interpolation:** The simplest method where the pixel takes the color of the closest texel. This is fast but often results in "blocky" or aliased images.

- **Bilinear Filtering:** Samples the four nearest texels and averages their colors based on distance, creating a smoother transition.
- **Anisotropic Filtering:** Advanced filtering that improves quality when viewing textures at sharp angles (oblique viewing), preventing blurring in the distance.

The Difference: Bilinear vs. Anisotropic

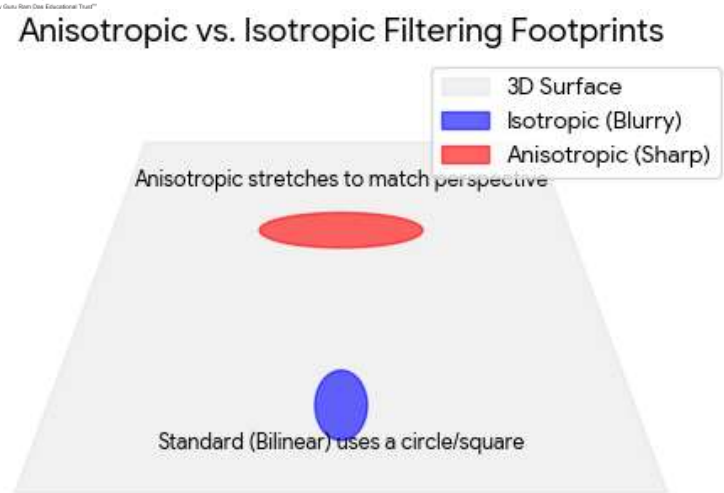
Feature	Bilinear/Trilinear (Isotropic)	Anisotropic
Assumption	Pixel footprint is always a **Square**.	Pixel footprint can be an **Ellipse/Rectangle**.
Averaging	Samples a 2x2 or 4x4 square area.	Samples along the **axis of tilt** (line of sight).
Result	Textures look blurry in the distance.	Textures remain sharp even at sharp angles.
Cost	Very fast (low GPU hit).	Heavier (requires more texture samples).

The Algorithm

1. **Determine the Footprint:** Calculate the projection of the screen pixel onto the texture surface.
2. **Check for "Anisotropy":** If the footprint is stretched (one axis is much longer than the other), the system identifies the "direction of anisotropy."
3. **Define the Sampling Line:** Instead of a square, the GPU creates a line or a trapezoid along the longer axis.
4. **Multi-Sampling:** The GPU takes **multiple** bilinear/trilinear samples along this line (e.g., 2x, 4x, 8x, or 16x).
5. **Weighted Average:** It averages these multiple samples to produce the final pixel color.

Visualizing the "Stretched" Footprint

In the diagram below, notice how a square pixel on your screen becomes a long, thin shape when projected onto a tilted floor. Anisotropic filtering samples along that long shape.



3. Mipmapping

Mipmapping is a technique used to optimize rendering and reduce aliasing (flickering/shimmering) for distant objects.

- **The Concept:** The system pre-calculates several versions of the same texture at different resolutions (levels of detail). For example, 256 X 256 a texture might have versions at 128 X 128 , 64 X 64 and so on.
- **Trilinear Filtering:** This process smooths the transition between two different mipmap levels. It performs bilinear filtering on both levels and then interpolates between them to eliminate visible "lines" where the resolution changes.

Concept	Key Practical Focus
Texels	The individual units/pixels of a texture image.
Interpolation	Calculating (U,V) values for points between vertices.
Minification	When one pixel covers many texels (requires Mipmapping).
Magnification	When one texel covers many pixels (requires Bilinear filtering).

3. Common Mapping Techniques

1. **Planar Mapping**: Projects the texture like a slide projector onto one axis (e.g., ignoring
2. **Cylindrical/Spherical**: Wraps the texture around the object (useful for globes or trees).

3. **UV Unwrapping**: Manually flattening a 3D model into 2D "islands" so a single image can cover complex shapes without stretching.

4. **1. Texture Mapping & Coordinates**

   Texture mapping is the process of "pasting" a 2D image (called a **texture**) onto the surface of a 3D model.

- **Texels**: Just as a screen is made of pixels, a texture is made of **texture elements** or "texels".
- **UV Coordinates**: To determine which part of a 2D image belongs to which part of a 3D surface, we use a 2D coordinate system

2. Mapping Techniques

Different shapes require different projection methods to minimize distortion:

- **Planar Mapping**: Projects the texture along a single axis; best for flat surfaces like walls.
- **Cylindrical Mapping**: Wraps the texture around a central axis; ideal for tree trunks or columns.
- **Spherical Mapping**: Projects points radially from a center point onto a sphere.
- **Box/Cube Mapping**: Projects the texture onto the six faces of a cube; often used for skyboxes or environment reflections.

3. Texture Filtering

When a texture is rendered, its texels rarely match the screen pixels perfectly in size or angle. Filtering resolves these mismatches:

- **Magnification**: When one texel covers many pixels (zooming in), **Bilinear Filtering** averages the four nearest texels to prevent a "blocky" look.
- **Minification**: When many texels fit into a single pixel (zooming out), it can cause "shimmering" or aliasing.

4. Mipmapping

Mipmapping is the primary solution for minification aliasing.

1. Comparison Table: Texture Filtering & Mipmapping

This table is designed to help you quickly distinguish between the common techniques used in 3D rendering exams.

Feature	Nearest Neighbor	Bilinear Filtering	Mipmapping
Primary Goal	Fast, simple mapping	Smoothing "blocky" pixels	Reducing "shimmering" on distant objects

Logic	Picks the single closest texel to the pixel.	Averages 4 surrounding texels.	Uses pre-scaled smaller versions of the image.
Visual Result	Jagged, "pixelated" look when close up.	Smooth, blurred appearance when close up.	Sharp, flicker-free textures at a distance.
Performance	Highest (Very fast).	Medium (Requires 4 texel fetches).	Fast (Better cache usage, but 33% more memory).
When to Use	Retro-style games or UI.	Modern 2D/3D games.	Standard for all 3D environments.

Differences Between PPI and DPI

What is PPI?
PPI stands for Pixels Per Inch. PPI means the number of pixels in an inch of any digital image. It enhances the clarity, quality, resolution, and definition of the image on the screen. Having a higher PPI means that the print will be clearer. It refers to the resolution of any digital image.

Types of PPI:
PPI is of several types such as:

- **Screen PPI:** It is a measurement of pixels on the screen per inch that explains and defines the clarity and quality of the image on the device's screen.
- **Print PPI:** The printed image can only be clear and of good quality when the image or the print is made with high-resolution standards.
- **Scanner PPI:** It describes the scanning quality of the scanner.
- **Web Design PPI:** It plays an important role in web designing as the web pages can be accessed on different types of devices with various sizes of screens a laptop screen is bigger than a smartphone screen so the resolution needs to work according to both so that the website is clear on both the devices. We often call it the responsiveness of a website.

Uses of PPI:
- It is used in image editing.
- It is useful in Video editing.
- It can be experienced in VR and AR.
- E-books have proper readable texts and images for the ones who love to read.
- It plays an important role in User interface design(UI).
- It is useful in creating websites.

Advantages of PPI:
- High-quality images can be produced.
- An image's resolution can be improved.
- It results in better editing.

- It provides a clear image with sharp details.
- It helps in making the web page responsive.
- It gives better results in digital art on graphic tablets.
- Visuals become more attractive which has an effect on the eyes of the users.
- The increased level of detail makes the work more precise.
- It enhances the presentation of colours and shapes.
- It is helpful in video editing and graphics art.

Disadvantages of PPI:

- The application should be compatible to use on a certain device like smart phones, tablets, MacBooks, laptops, etc.
- We often say that the pixels got cracked, this is because of low PPI.
- The size of the files like images or graphics automatically increases and occupies more space in the memory of the system or gadgets.

What is DPI?
DPI stands for Dots Per Inch. DPI refers to the number of ink dots on a physically printed page. It shows the clarity of the image on the printed page and refers to the printed resolution on the page. Having a higher DPI means that the print will be sharper.

Types of DPI:
DPI is of several types such as:

- **Printer DPI:** It refers to the resolution of a printed image by any printer.
- **Scanner DPI:** Scanners use DPI to improve the quality and details of their scans.
- **Mouse DPI:** It refers to the movement of mouse cursor on the screen of the computer.
- **Monitor DPI:** It refers to the resolution of laptop or computer monitors.

Uses of DPI:

- It is used for Printing and photocopying.
- Scanning tasks can be clear.
- Graphic tablets
- It is useful in photography and videography.
- It is used in interior designs like wallpapers, PVC etc.
- Digital arts need finishing of details which needs a good understanding and usage of dots on a page.

Advantages of DPI:

- High-quality printouts can be produced
- Precise printing is done.
- Printed texts can be properly read.
- It gives better quality photocopies.
- It makes your document look formal and professional.
- It gives proper effects of colours.
- It helps in photography.
- Arts might look like real.
- It gives proper differentiation in colours on the print.
- Clear details of printed images can be seen and noticed easily which enhances the beauty of the work done.

Disadvantages of DPI:

- A low DPI output can result in a blurry image (non-clear image) on the printed paper or on the screen.
- Costs of print-outs might increase due to the high resolution of images.
- Little knowledge of DPI can result in unsatisfactory results on paper.

PPI	DPI
It is related to digital images.	It is related to printed image.
Higher PPI indicates that the picture is more clear.	A higher DPI indicates that the printed picture is of high quality.
It is measured in pixels per inch.	It is measured in dots per inch.
It represents the size of pixels of a digital image.	It represents the size of dots of a printed image.
These pictures can be seen on mobiles, tablets, laptops, etc.	These pictures can be accessed through printers.
It is termed for the quality of pictures on the screen.	It is termed for the quality of pictures on the paper.

<u>Java Program for polygon and color</u>

```
import java.awt.*;

import javax.swing.*;

public class SimpleHut extends JPanel {

    @Override

    public void paintComponent(Graphics g) {

        super.paintComponent(g);

        // Wall (Rectangle)
```

```
        g.setColor(new Color(210, 180, 140)); // Tan color
        g.fillRect(150, 200, 200, 150);
        // Roof (Polygon/Triangle)
        g.setColor(new Color(139, 69, 19)); // Brown
        int[] xRoof = {130, 250, 370};
        int[] yRoof = {200, 100, 200};
        g.fillPolygon(xRoof, yRoof, 3);
         // Door
        g.setColor(new Color(101, 67, 33));
        g.fillRect(225, 270, 50, 80);
        // Window
        g.setColor(Color.CYAN);
        g.fillRect(170, 230, 40, 40);
        g.setColor(Color.BLACK);
        g.drawRect(170, 230, 40, 40);
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame("Simple Hut");
        frame.add(new SimpleHut());
        frame.setSize(500, 500);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```